

Chapter 1

Exercise 1 (Conceptual)

Statement. For each of parts (a) through (d), indicate whether we would generally expect the performance of a flexible statistical learning method to be better or worse than an inflexible method. Justify your answer.

- (a) The sample size n is extremely large, and the number of predictors p is small.
- (b) The number of predictors p is extremely large, and the number of observations n is small.
- (c) The relationship between the predictors and response is highly non-linear.
- (d) The variance of the error terms, i.e. $\sigma^2 = \text{Var}(\varepsilon)$, is extremely high.

Solution

- (a) **Flexible is generally better.** With *very large* n and *small* p , flexible methods can estimate complex patterns while keeping variance under control (because abundant data stabilizes the fit). With enough observations, the risk of overfitting is reduced, so capturing finer structure can improve predictive accuracy.
- (b) **Flexible is generally worse (inflexible better).** When p is extremely large but n is small, flexible methods typically have *high variance* and can overfit heavily (many degrees of freedom relative to the data). Inflexible methods (or strong regularization / dimension reduction) tend to generalize better in this regime.
- (c) **Flexible is generally better.** If the true relationship is highly non-linear, an inflexible method (e.g., a straight line) suffers high *bias*. A flexible method is designed to model non-linear structure, reducing bias and often improving performance.
- (d) **Flexible is generally worse (inflexible better).** With extremely high noise variance σ^2 , the data contain a lot of randomness. Flexible methods are more prone to chasing noise (again: higher variance), whereas inflexible methods tend to smooth over noise and can yield better generalization.

Exercise 9 (Auto Data Set)

Statement. This exercise involves the **Auto** data set studied in the lab. Make sure that the missing values have been removed from the data.

- (a) Which of the predictors are quantitative, and which are qualitative?

- (b) What is the range of each quantitative predictor?
- (c) What is the mean and standard deviation of each quantitative predictor?
- (d) Now remove the 10th through 85th observations. What is the range, mean, and standard deviation of each predictor in the subset of the data that remains?
- (e) Using the full data set, investigate the predictors graphically, using scatterplots or other tools of your choice. Create some plots highlighting the relationships among the predictors. Comment on your findings.
- (f) Suppose that we wish to predict gas mileage (**mpg**) on the basis of the other variables. Do your plots suggest that any of the other variables might be useful in predicting **mpg**? Justify your answer.

Solution

(a) Quantitative vs. qualitative

In the standard Auto data set (as used in many introductory labs), the variables are:

`mpg`, `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `year`, `origin`, `name`.

- **Quantitative predictors:** `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `year`. (These are numeric measurements; `cylinders` is an integer but still quantitative.)
- **Qualitative predictors:** `name` (a string label), and typically `origin` (even though it may be coded as 1/2/3, it represents categories such as regions of manufacture and is commonly treated as categorical).
- **Response:** `mpg` (quantitative).

(b) Range of each quantitative predictor

Compute min and max for each quantitative predictor. For example, in Python with pandas:

```
import pandas as pd

df = pd.read_csv("Auto.csv")          # adjust path/name as needed
df = df.dropna()                      # remove missing values (or use
    specific parsing rules)

quant = ["cylinders", "displacement", "horsepower", "weight", "acceleration", "
    year"]
ranges = pd.DataFrame({
    "min": df[quant].min(),
    "max": df[quant].max()
})
ranges["range"] = ranges["max"] - ranges["min"]
print(ranges)
```

(c) Mean and standard deviation of each quantitative predictor

Compute sample means and sample standard deviations:

```
summary = pd.DataFrame({
    "mean": df[quant].mean(),
    "std": df[quant].std(ddof=1)    # sample standard deviation
})
print(summary)
```

(d) Remove the 10th through 85th observations; recompute summaries

Interpret “10th through 85th observations” as removing rows 10–85 inclusive in *1-based* indexing. In Python’s *0-based* indexing, that corresponds to indices 9 through 84 inclusive.

```
df_sub = df.drop(df.index[9:85])    # drops indices 9..84

ranges_sub = pd.DataFrame({
    "min": df_sub[quant].min(),
    "max": df_sub[quant].max()
})
ranges_sub["range"] = ranges_sub["max"] - ranges_sub["min"]

summary_sub = pd.DataFrame({
    "mean": df_sub[quant].mean(),
    "std": df_sub[quant].std(ddof=1)
})

print(ranges_sub)
print(summary_sub)
```

(e) Graphical investigation and comments

Useful plots include:

- **Scatterplot matrix** (pairwise scatterplots) for the quantitative variables.
- **Correlation heatmap** among quantitative predictors (and with mpg).
- **Boxplots of mpg by origin** (categorical differences).

Example code:

```
import matplotlib.pyplot as plt
import pandas as pd

# Correlation heatmap (simple)
corr = df[["mpg"] + quant].corr()
plt.imshow(corr, aspect="auto")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.index)), corr.index)
plt.title("Correlation heatmap")
plt.tight_layout()
```

```
plt.show()

# A few targeted scatterplots:
plt.scatter(df["weight"], df["mpg"])
plt.xlabel("weight"); plt.ylabel("mpg"); plt.title("mpg vs weight")
plt.show()

plt.scatter(df["horsepower"], df["mpg"])
plt.xlabel("horsepower"); plt.ylabel("mpg"); plt.title("mpg vs horsepower")
plt.show()
```

Typical findings (what you usually see in these plots):

- **mpg** tends to **decrease** as **weight**, **displacement**, **horsepower**, and **cylinders** increase (strong negative associations).
- Many predictors are **strongly correlated with each other**: e.g., heavier cars often have more cylinders and larger displacement, and also tend to have higher horsepower. This indicates **multicollinearity**.
- **year** often shows a **positive** association with **mpg**, consistent with improved fuel efficiency in later model years.
- **origin** categories often show different central tendencies for **mpg** (visible via boxplots).

(f) Predicting mpg: which variables look useful?

Yes. The plots typically suggest that several variables are useful predictors of **mpg**:

- **Very useful**: **weight**, **horsepower**, **displacement**, **cylinders** (often strong, mostly monotone negative relationships with **mpg**).
- **Useful**: **year** (often a clear positive trend with **mpg**).
- **Potentially useful**: **origin** (different groups can have systematically different **mpg** distributions).
- **Less clear / weaker**: **acceleration** may have a weaker relationship and can depend on the sample; it can still add incremental predictive value.

A key modeling note suggested by the plots is that because predictors like **weight**, **displacement**, **horsepower**, and **cylinders** are mutually correlated, one may not need all of them simultaneously in a simple linear model; regularization or careful model selection can help.

Note on reproducibility: the numerical answers in parts (b)–(d) depend on the exact version of the **Auto** file and how missing values are handled. The code above computes the required quantities directly from your local data set.